



データベース講座【基礎編】 設計力を身につける ～テーブル設計・ER図・SQL応用～ 1回目

2026/06/30

Kazuma Sekiguchi

自己紹介



関口和真

株式会社コムセントCTO

Webシステム開発、スマートフォンアプリ制作、
サーバー構築、運用など

スマートフォンを使った動画配信アプリの制作
サーバーサイドシステムの作成
フロントエンド部分の作成

目標

- データベースにおけるテーブル設計が理解できる
- ER図の書き方、読み方が理解できる
- 複数のテーブルを組み合わせた情報の取得方法を理解できる
- 入門編を知識のベースとしています
- 全部の講座を受講いただくとPHP+MySQLを通じたシステムの作成まで行ないます

今回のAgenda

- 主キーの役割が分かる
- 外部キーの考え方の役割
- テーブル分割の意味
- 後半ではphpmyadminを利用して実際にテーブルの関係を確認する
 - 次回はVSCodeなどを利用します
 - 拡張機能のDraw.io Integrationを使うので準備を

テーブル設計

- テーブルは1つのデータベースに対して複数存在する
- テーブルは思いつきで作成すると無駄が生じる
- 計画的に作成する必要がある
- 主として以下を意識してテーブルを設計する
 - データベースにどのようなテーブルを作るか考えること
 - 各テーブルにどのようなカラムを持たせるか考えること
 - 1件のレコードが何を表すか決めること
 - テーブル同士の関係を考えること
 - データの重複や矛盾を減らすこと
 - 後から検索、更新、集計しやすい形にすること

テーブル設計の重要性

- データベースは作れば終わりではない
 - 後からデータを追加、検索、更新、削除する
 - アプリケーションから何度も利用される
- テーブル構造が悪いと、SQLが複雑になる
- データの重複が増える
- 更新漏れや不整合が起きやすくなる
- 機能追加のたびに修正が難しくなる
- DBのパフォーマンスが落ちる

会員と投稿の例

- 会員情報

- 会員ID
- 氏名
- メールアドレス
- 会員状態
- 登録日時

- 投稿情報

- 投稿ID
- 投稿タイトル
- 投稿本文
- 投稿日時
- 投稿者

すべてを1つのまとめる

- 力業

post_id	title	body	user_name	user_email
1	初投稿	はじめまして	Smith	smith@example.com
2	今日の記録	今日の内容	Smith	smith@example.com
3	お知らせ	お知らせ内容	Brown	brown@example.com

- 一応、理解はできる

- ただしデータ数が増えると混乱の元になる

1つの表にまとめた場合の問題

- 投稿が増えるたびに、会員名とメールアドレスを繰り返し保存する
 - データ量が増えるだけで無駄が多い
- メールアドレスを変更する場合、複数行を更新する必要がある
 - 一部だけ更新漏れすると、同じ人なのに違うメールアドレスが残る
- 会員情報と投稿情報が混ざっている
 - 1件のレコードが何を表すのか分かりにくくなる

データの重複とは

- 同じ意味のデータが複数箇所に保存されている状態
 - 同じ会員名が複数行にある
 - 同じメールアドレスが複数行にある
- 少量なら問題に見えにくい
 - 件数が増えると管理が難しくなる
 - データも無駄に増えるため、パフォーマンスにも影響が出る
- 更新漏れや表記ゆれの原因になる

主キー

- テーブル内の1レコードを一意に識別するためのカラム
- 英語ではPrimary Key (PKと略すことも)
- 同じ値が重複しない
 - 必ず違う値になる必要がある
- NULLにならない
 - NULL許可できない
- 多くの場合、idカラムを主キーにする
- レコードを正確に指定するために使う
- UPDATEやDELETEで対象を絞るときにも重要

なぜ主キーが必要か

- 同じ名前の人がいる可能性がある
 - 同姓同名
- 人間には同じに見える情報でも、別レコードとして扱う必要がある
 - 商品名が同じ商品
 - タイトルが同じ投稿
- 確実に1件を指定するために主キーが必要
 - 名前やタイトルのように変更される可能性がある値は、主キーに向かない

主キーを持たせることで

- UPDATEやDELETE時に主キーを指定することで、確実に1件のデータを指定、特定することが可能
 - 主キーは異なる値を有するため、確実に特定の1件にできる
- 学籍番号や社員番号もある種の主キー
- メールアドレスは主キーになり得るが通常しない
 - 通常は数字にした方が主キーとしては使いやすい
 - 連番を付けていくことで、重複のないデータを主キーとして利用する

AUTO_INCREMENTとは

- MySQLで自動採番を行う仕組み
- レコードを追加するたびに、自動で番号が増える
- 1, 2, 3, 4 のような連番になる
 - 連番になるため、同じ値は使われることが無い
- 主キーのidカラムでよく使われる
 - 主キー以外でも使うことはできるが、ほぼ主キー目的
- 初心者向けには最も理解しやすい主キーの作り方
- 通常の学習用・小規模Webアプリでは扱いやすい
 - 比較的大規模でも使う（億単位でも特に問題は起きないため）

UUID

- UUIDは、重複しにくい長いID
- 例：550e8400-e29b-41d4-a716-446655440000
- MySQLでは UUID() 関数で生成できる
- 外部公開用IDや分散環境で使われることがある
- AutoIncrementは連番故に推測が可能というデメリットが生じる
 - UUIDは連番ではないので、推測は不可能に近くなる
 - データベースに依存すること無く、IDを設定することが可能
- それなりにデメリットもあるので、使い分けるのが一般的

テーブルを分ける考え方

- 会員情報と投稿情報は別の種類のデータ
- データ種類ごとにテーブルを分ける、というのが基本
- 別の種類のデータは、別テーブルに分けることを考える
 - 会員情報は会員テーブルに保存する
 - 投稿情報は投稿テーブルに保存する
- 投稿テーブルには、投稿者を表す値を持たせる
 - 同じ会員情報を投稿ごとに繰り返し保存しない

データ種類でテーブルを分割

id	user_id	title	body
1	1	初投稿	はじめまして
2	1	今日の記録	今日の内容
3	2	お知らせ	お知らせ内容

postsテーブル

usersテーブル

id	user_name	user_email
1	Smith	smith@example.com
2	Brown	brown@example.com

テーブルを分けると

- データを種類ごとにテーブルに分けて保存する
 - 会員情報はusersテーブルに保存
 - 投稿情報はpostsテーブルに保存
- 投稿者名やメールアドレスはpostsに持たせない
 - postsには、投稿者を示すuser_idを持たせる
 - user_id により、どの会員の投稿が分かる
- 会員情報を修正する場合、usersの1行を直せばよい

外部キーとは

- 別のテーブルのレコードを参照するためのカラム
- 英語ではForeign Key (FKとも表す)
- あるテーブルと別のテーブルの関係を表すためのカラム
 - postsテーブルのuser_idは、usersテーブルのidを参照する
 - どの投稿がどの会員のものを表せる
 - テーブル分割後の関係をつなぐ役割を持つ

主キーと外部キーの関係

- usersテーブルのidは主キー
- postsテーブルのuser_idは外部キー
- 外部キーは、別テーブルの主キーを参照することが多い
 - 主キーは参照される側
 - 外部キーは参照する側
 - この関係により、複数テーブルを関連付けられる

id	user_id	title	body
1	1	初投稿	はじめまして
2	1	今日の記録	今日の内容
3	2	お知らせ	お知らせ内容



id	user_name	user_email
1	Smith	smith@example.com
2	Brown	brown@example.com

外部キー制約

- 外部キーには、考え方としての外部キーと、DBに設定する外部キー制約がある
 - 外部キー制約を設定すると、存在しない親データを参照しにくくなる
- 存在しないuser_idをpostsに入れることを防げる
 - もし、存在しないuser_idをいれようとするエラーになる
 - データの整合性を保ちやすくなる
- 例えば、postsテーブルに存在しないuser_idのデータが入ってきたら整合性がおかしくなる
 - 外部キー制約を付けることで、この不整合を防ぐことができる

1対多の関係

- 1つのデータに対して、複数のデータが紐づく関係
 - 会員1人が複数の投稿を持つ
 - カテゴリ1つに複数の商品が属する
 - 注文1件に複数の注文明細がある
 - 投稿1件に複数のコメントが付く
- リレーショナルデータベースでよく出てくる基本的な関係
- 多対多という関係もあるし、1対1という関係もある

1対多と外部キー

- 1対多の関係では、多側のテーブルに外部キーを持たせることが多い
 - 会員と投稿なら、投稿側にuser_idを持たせる
 - カテゴリと商品なら、商品側にcategory_idを持たせる
 - 注文と注文明細なら、注文明細側にorder_idを持たせる
- 多側から1側を参照することで、関係を表せる

ありがとうございました。

また次回